

Cryptography Lab Programs

Implementation, Testing & Documentation of Classical Ciphers

1. Caesar Cipher
2. Monoalphabetic Cipher
3. Polyalphabetic Cipher (Vigenere)
4. Hill Cipher
5. Playfair Cipher
6. Rail Fence Cipher
7. Double Columnar Transposition Cipher

GitHub Repository: <https://github.com/thelivinbot/Cryptography-Practical-5780->

Program 1: Caesar Cipher

Encryption & Decryption Implementation

Aim

To implement a Python program for encryption and decryption of a message using the Caesar Cipher technique, where each letter in the plaintext is shifted by a fixed number of positions (key) in the alphabet.

Source Code (Python)

```
def encrypt(text, key):
    result = ""

    for char in text:
        if char.isalpha():
            if char.isupper():
                result += chr((ord(char) - ord('A') + key) % 26 + ord('A'))
            else:
                result += chr((ord(char) - ord('a') + key) % 26 + ord('a'))
        else:
            result += char

    return result

def decrypt(text, key):
    return encrypt(text, -key)

text = input("Enter message: ")
key = int(input("Enter key: "))

encrypted = encrypt(text, key)
print("Encrypted message:", encrypted)

decrypted = decrypt(encrypted, key)
print("Decrypted message:", decrypted)
```

Sample Input / Output

Enter message: GITAM UNIVERSITY
Enter key: 6

Encrypted message: MOZGS ATOBKXYOZE
Decrypted message: GITAM UNIVERSITY

Program 2: Monoalphabetic Cipher

Encryption & Decryption Implementation

Aim

To implement a Python program for encryption and decryption of a message using the Monoalphabetic Cipher technique, where each letter of the plaintext alphabet is mapped to a unique letter of a scrambled 26-letter key.

Source Code (Python)

```
def encrypt(text, key):
    alphabet = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
    result = ""

    for char in text.upper():
        if char.isalpha():
            index = alphabet.index(char)
            result += key[index]
        else:
            result += char

    return result

def decrypt(text, key):
    alphabet = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
    result = ""

    for char in text.upper():
        if char.isalpha():
            index = key.index(char)
            result += alphabet[index]
        else:
            result += char

    return result

text = input("Enter message: ")
key = input("Enter 26-letter key: ").upper()

encrypted = encrypt(text, key)
print("Encrypted message:", encrypted)

decrypted = decrypt(encrypted, key)
print("Decrypted message:", decrypted)
```

Sample Input / Output

```
Enter message: GITAM UNIVERSITY
Enter 26-letter key: QWERTYUIOPASDFGHJKLZXCVBNM

Encrypted message: UOZQD XFOCTKLOZN
Decrypted message: GITAM UNIVERSITY
```

Program 3: Polyalphabetic Cipher (Vigenere)

Encryption & Decryption Implementation

Aim

To implement a Python program for encryption and decryption of a message using the Polyalphabetic (Vigenere) Cipher technique, where each letter of the plaintext is shifted according to the corresponding repeating letter of a keyword.

Source Code (Python)

```
def encrypt(text, key):
    result = ""
    key = key.upper()
    key_index = 0

    for char in text.upper():
        if char.isalpha():
            shift = ord(key[key_index % len(key)]) - ord('A')

            encrypted = chr(
                (ord(char) - ord('A') + shift) % 26 + ord('A')
            )

            result += encrypted
            key_index += 1
        else:
            result += char

    return result

def decrypt(text, key):
    result = ""
    key = key.upper()
    key_index = 0

    for char in text.upper():
        if char.isalpha():
            shift = ord(key[key_index % len(key)]) - ord('A')

            decrypted = chr(
                (ord(char) - ord('A') - shift) % 26 + ord('A')
            )

            result += decrypted
            key_index += 1
        else:
            result += char

    return result

text = input("Enter message: ")
key = input("Enter key: ")

encrypted = encrypt(text, key)
print("Encrypted message:", encrypted)

decrypted = decrypt(encrypted, key)
print("Decrypted message:", decrypted)
```

Sample Input / Output

Enter message: GITAM UNIVERSITY

Enter key: KEY

Encrypted message: QMRAY EMFVANWMTY

Decrypted message: GITAM UNIVERSITY

Program 4: Hill Cipher

Encryption & Decryption Implementation

Aim

To implement a Python program for encryption and decryption of a message using the Hill Cipher technique, which uses matrix multiplication (a 2x2 key matrix) over modulo 26 to encrypt pairs of letters, and the modular inverse of the key matrix to decrypt.

Source Code (Python)

```
def encrypt(text, key):
    text = text.upper().replace(" ", "")

    if len(text) % 2 != 0:
        text += "X"

    result = ""

    for i in range(0, len(text), 2):
        x1 = ord(text[i]) - ord('A')
        x2 = ord(text[i + 1]) - ord('A')

        y1 = (key[0][0] * x1 + key[0][1] * x2) % 26
        y2 = (key[1][0] * x1 + key[1][1] * x2) % 26

        result += chr(y1 + ord('A'))
        result += chr(y2 + ord('A'))

    return result

def decrypt(text, key):
    determinant = (
        key[0][0] * key[1][1]
        - key[0][1] * key[1][0]
    ) % 26

    determinant_inverse = pow(determinant, -1, 26)

    inverse_key = [
        [
            (key[1][1] * determinant_inverse) % 26,
            (-key[0][1] * determinant_inverse) % 26
        ],
        [
            (-key[1][0] * determinant_inverse) % 26,
            (key[0][0] * determinant_inverse) % 26
        ]
    ]

    return encrypt(text, inverse_key)

text = input("Enter message: ")

key = [
    [3, 3],
    [2, 5]
]

encrypted = encrypt(text, key)
```

```
print("Encrypted message:", encrypted)

decrypted = decrypt(encrypted, key)
print("Decrypted message:", decrypted)
```

Sample Input / Output

Enter message: GITAM UNIVERSITY

Encrypted message: QAFMSULOXKBUDHLH

Decrypted message: GITAMUNIVERSITYX

Program 5: Playfair Cipher

Encryption & Decryption Implementation

Aim

To implement a Python program for encryption and decryption of a message using the Playfair Cipher technique, which encrypts digraphs (letter pairs) using a 5x5 key matrix generated from a keyword, following row, column, and rectangle rules.

Source Code (Python)

```
def create_matrix(key):
    key = key.upper().replace("J", "I")
    alphabet = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"

    result = ""

    for char in key + alphabet:
        if char not in result and char.isalpha():
            result += char

    matrix = []

    for i in range(0, 25, 5):
        matrix.append(list(result[i:i + 5]))

    return matrix

def find_position(matrix, char):
    for row in range(5):
        for col in range(5):
            if matrix[row][col] == char:
                return row, col

def prepare_text(text):
    text = text.upper().replace("J", "I")
    text = "".join(char for char in text if char.isalpha())

    result = ""
    i = 0

    while i < len(text):
        a = text[i]

        if i + 1 < len(text):
            b = text[i + 1]

            if a == b:
                result += a + "X"
                i += 1
            else:
                result += a + b
                i += 2
        else:
            result += a + "X"
            i += 1

    return result

def encrypt(text, matrix):
    result = ""
```

```

for i in range(0, len(text), 2):
    a = text[i]
    b = text[i + 1]

    r1, c1 = find_position(matrix, a)
    r2, c2 = find_position(matrix, b)

    if r1 == r2:
        result += matrix[r1][(c1 + 1) % 5]
        result += matrix[r2][(c2 + 1) % 5]

    elif c1 == c2:
        result += matrix[(r1 + 1) % 5][c1]
        result += matrix[(r2 + 1) % 5][c2]

    else:
        result += matrix[r1][c2]
        result += matrix[r2][c1]

return result

def decrypt(text, matrix):
    result = ""

    for i in range(0, len(text), 2):
        a = text[i]
        b = text[i + 1]

        r1, c1 = find_position(matrix, a)
        r2, c2 = find_position(matrix, b)

        if r1 == r2:
            result += matrix[r1][(c1 - 1) % 5]
            result += matrix[r2][(c2 - 1) % 5]

        elif c1 == c2:
            result += matrix[(r1 - 1) % 5][c1]
            result += matrix[(r2 - 1) % 5][c2]

        else:
            result += matrix[r1][c2]
            result += matrix[r2][c1]

    return result

key = input("Enter key: ")
text = input("Enter message: ")

matrix = create_matrix(key)

print("\nPlayfair Matrix:")
for row in matrix:
    print(" ".join(row))

prepared = prepare_text(text)

encrypted = encrypt(prepared, matrix)
print("\nEncrypted message:", encrypted)

decrypted = decrypt(encrypted, matrix)
print("Decrypted message:", decrypted)

```

Sample Input / Output

Enter key: MONARCHY
Enter message: STEGANOGRAPHY

Playfair Matrix:

M	O	N	A	R
C	H	Y	B	D
E	F	G	I	K
L	P	Q	S	T
U	V	W	X	Z

Encrypted message: TLFIRANFMRVFBW
Decrypted message: STEGANOGRAPHYX

Program 6: Rail Fence Cipher

Encryption & Decryption Implementation

Aim

To implement a Python program for encryption and decryption of a message using the Rail Fence Cipher technique, which writes the plaintext in a zig-zag pattern across a given number of rails and reads it off row by row to form the ciphertext.

Source Code (Python)

```
def encrypt(text, key):
    rail = ['' for _ in range(key)]
    row = 0
    direction = 1

    for char in text:
        rail[row] += char

        if row == 0:
            direction = 1
        elif row == key - 1:
            direction = -1

        row += direction

    return ''.join(rail)

def decrypt(cipher, key):
    pattern = []
    row = 0
    direction = 1

    for _ in cipher:
        pattern.append(row)

        if row == 0:
            direction = 1
        elif row == key - 1:
            direction = -1

        row += direction

    rail = [''] * key
    index = 0

    for r in range(key):
        count = pattern.count(r)
        rail[r] = list(cipher[index:index + count])
        index += count

    result = ""

    for r in pattern:
        result += rail[r].pop(0)

    return result

text = input("Enter message: ")
key = int(input("Enter number of rails: "))
```

```
encrypted = encrypt(text, key)
print("Encrypted message:", encrypted)
```

```
decrypted = decrypt(encrypted, key)
print("Decrypted message:", decrypted)
```

Sample Input / Output

Enter message: GITAM UNIVERSITY

Enter number of rails: 3

Encrypted message: GMISIA NVRIYTUET

Decrypted message: GITAM UNIVERSITY

Program 7: Double Columnar Transposition Cipher

Encryption & Decryption Implementation

Aim

To implement a Python program for encryption and decryption of a message using the Double Columnar Transposition Cipher technique, which applies the columnar transposition process twice, using two separate keys, to increase the strength of the encryption.

Source Code (Python)

```
def columnar_transposition(text, key):
    columns = len(key)
    rows = (len(text) + columns - 1) // columns

    text += 'X' * (rows * columns - len(text))

    matrix = []

    for i in range(0, len(text), columns):
        matrix.append(list(text[i:i + columns]))

    order = sorted(range(columns), key=lambda x: key[x])

    result = ""

    for col in order:
        for row in range(rows):
            result += matrix[row][col]

    return result

def inverse_columnar_transposition(text, key):
    columns = len(key)
    rows = len(text) // columns

    order = sorted(range(columns), key=lambda x: key[x])

    matrix = [[''] * columns for _ in range(rows)]

    index = 0

    for col in order:
        for row in range(rows):
            matrix[row][col] = text[index]
            index += 1

    result = ""

    for row in range(rows):
        for col in range(columns):
            result += matrix[row][col]

    return result

def encrypt(text, key1, key2):
    first = columnar_transposition(text, key1)
    second = columnar_transposition(first, key2)

    return second
```

```
def decrypt(text, key1, key2):
    first = inverse_columnar_transposition(text, key2)
    second = inverse_columnar_transposition(first, key1)

    return second

text = input("Enter message: ")
key1 = input("Enter first key: ")
key2 = input("Enter second key: ")

encrypted = encrypt(text, key1, key2)
print("Encrypted message:", encrypted)

decrypted = decrypt(encrypted, key1, key2)
print("Decrypted message:", decrypted)
```

Sample Input / Output

```
Enter message: GITAMUNIVERSITY
Enter first key: ZEBRA
Enter second key: TIGER
```

```
Encrypted message: TSUYNGEITIARMIV
Decrypted message: GITAMUNIVERSITY
```